

Review packet

RL for a lab-automation digital twin - the done work, in five hops

What this is

The whole record of what has been done, as five hops in order. No sixth file is needed, and none of the learning tracks are on the list.

	Hop	Source	Lines	Read for
1	Where the project stands	CLAUDE.md	220	Current state, then Roadmap. Start here so the rest has a frame.
2	Every measured number and every code defect	_docs/PUBLICATION_MATERIALS.md	327	The findings index, then the entries you care about.
3	What was done, round by round	_docs/WORKLOG.md	150	The index table, then the round entries.
4	Scope, and the architecture choice still open	_docs/strategy.md	292	Parts 2 and 5.
5	Present, ask, propose	_docs/team-discussion.md	232	All of it - it is short.

Two things to expect while reading. Hops 2 and 3 overlap by design: the worklog says what was done, publication materials says what is worth telling someone else. And the fifth environment (AdroitHandDoor-v1) is marked as an estimate awaiting a measured result in exactly one place, in hop 3 - if you see that marker, it has not been replaced yet.

Not included, and deliberately. The three round files under _docs/rounds/ (838 lines) are provenance - open them in the repo when a specific claim needs backing, not as part of this read. _docs/decisions.md is not one of the five hops either; hops 4 and 5 cite the decisions that matter. _docs/environment.md is setup, not record.

Generated 2026-08-21 from RL_DT@29f9573. Assembled from the markdown sources named in each section header; nothing was rewritten for this packet. Chromium print does not support page numbers, so the section banners are the navigation.

Hop 1 of 5 — where the project stands

RL_DT/CLAUDE.md - 221 lines

The review list calls for section Current state, then section Roadmap. The rest is included because this file is the map and it is short.

RL_DT — RL for a lab-automation digital twin

Working branch: `develop` · Remote: `https://tanuki.pnnl.gov/dewei.wang/rl-dt.git`

Paths are machine-specific — see `_docs/environment.md` for the current machine. On Dewei's PNNL workstation the project root is `C:\Users\wang109\OneDrive - PNNL\Research_OneDrive\RL_Twin`, with the repo at `... \RL_Twin\RL_DT`. (A `Z:` drive alias for this path existed on 2026-08-03 but is **not** stable — never hardcode `Z:.\`)

Documentation — everything is in `_docs/`

Start at `_docs/README.md`, which indexes the folder. 16 files, grouped by lifetime:

- **Live (6)** — `WORKLOG.md` (the round index), `PUBLICATION_MATERIALS.md` (every measured number and code defect — the canonical home; other files link to it), `decisions.md`, `strategy.md`, `team-discussion.md` (all open questions, grouped by who can answer them), `environment.md`.
- **History (3)** — `rounds/`, one file per round. Written once.
- **Learn (6)** — `learn/`: four numbered tracks (RL and TD3 · the toolkit · Isaac and `rl_games` · the door task), the one runnable file `td3_pendulum_demo.py`, and a `README.md` with the reading order.

`_hist/` at the repo root is where `/trace` writes verified work — see `_hist/README.md`. Reorganized 2026-08-19 (36 files → 16; the deletion mapping is in `_docs/WORKLOG.md`). `_notes/`, `_docs/archive/`, `_docs/meetings/`, `_docs/learning/` and `_docs/setup/` no longer exist.

Two repos, and they never mix. Toolkit code goes in `<PROJECT_ROOT>/SciOptControlToolkit` (a live clone, branch `develop-dw`); notes and logs go here. Never put notes in the toolkit, never put toolkit code here. **Never git push in either** — Dewei handles authentication.

Outside the repo and NOT version-controlled: `<PROJECT_ROOT>/_reference` — `digital_twin_models-main` (Isaac Sim USD assets + a preliminary door task), papers, and an older copy of the toolkit. ~2 GB. Never copy USD assets or PDFs into this repo; on a new machine re-obtain it before running anything Isaac-related — see `_docs/environment.md`.

Note: paths written as `_reference/...` sit one level *above* the repo.

Roadmap

Short term (now)

- [x] **Port TD3 from TensorFlow/Keras to PyTorch** — `TorchTD3-v0` landed 2026-08-17, 82 tests, additive only. SAC and DDPG not started.
- [x] **Validate the port in Gymnasium** — matched the Keras baseline on four environments; see `_docs/PUBLICATION_MATERIALS.md`. Still single-seed per arm.
- [] **≥3 seeds per arm with reported variance** — the largest remaining weakness in the evidence.
- [] **Prepare for Isaac Sim / Isaac Lab container integration** — keep the agent/buffer/model layer free of any Isaac dependency so it can be dropped into a container later. Blocker found: `agents/__init__.py` and

`run_continuous.py` import TensorFlow at module level, so the package will not import in a torch-only container (`_docs/strategy.md` Part 2).

Medium term

- [] Stand up the door-opening task as a proper Isaac Lab env, adapted from `_reference/digital_twin_models-main/RL_CloudTesting/` (preliminary — treat as a starting point, not a baseline to preserve).
- [] Add force/torque + contact observations and a force/torque penalty (the brief's "no brute-force pushing" requirement; currently absent).
- [] Add domain randomization — approach angle, handle rotation, base pose (currently all ranges are zero, so nothing generalizes yet).
- [] Reduce reliance on script-guided trajectory-tracking rewards; generalize the shaping.
- [] Reproducible runs: fixed seeds, ≥ 3 seeds, reported variance.

Long term

- [] A modular test bed: swap algorithm / task / scene by config, no code edits.
- [] Second task (tube-in-rack — asset exists at `_reference/digital_twin_models-main/ReinforcementLearning/TubeRacking.usd`) to substantiate the "framework" claim.
- [] Optional research upside: physics-informed / interpretable component (SINDy-style), carried from the group's accelerator work into robotics.
- [] Paper — see `_docs/learn/4-door-task.md` Part 3 for the three candidate theses and the skeleton.

Open questions

All of them, ranked and grouped by **who can answer**, live in `_docs/team-discussion.md` Part 2 — project scope, the RL toolkit, the digital twin, team-wide, plus the list of questions our own work already answered. Do not start a second list here.

The four that most change what gets built: what goes in the brief's blank Section 1.2 · the success criterion and tolerance (existing code commits to 180°) · the randomization envelope (currently all reset ranges are zero) · concrete force/torque limits (currently no force or torque anywhere in the task). Two unresolved forks that get expensive if decided late: **2F-85 vs 2F-140** gripper, and which scene is canonical.

Goal

Build a reusable, modular RL test bed for **human-centric lab automation** — robots operating equipment designed for human hands, under variable geometry. First task: a mobile manipulator (Clearpath Ridgeback + UR5e + Robotiq gripper) opens the door of an **Opentrons Flex** liquid handler inside an **Isaac Sim** digital twin, generalizing over randomized approach angles and handle rotations.

Two constraints from the brief drive design: **no reward hacking**, and **no brute-force pushing** (force/torque limits). Keep success metrics separate from training reward.

Team: Alvika leads the RL work; Dewei supports. DT assets from `prat615`. PM / toolkit author: Malachi Schram.

Stack

- **Python 3.10/3.11, PyTorch** (the reference toolkit is TF/Keras — we port, we don't wrap)
- **Gymnasium** for the local algorithm-validation track
- **NVIDIA Isaac Sim 5.1 + Isaac Lab** for the robot task (ManagerBasedRLEnv)
- **rl_games PPO** is the committed RL library for the Isaac Lab track
- **TensorBoard** for logging
- Hardware: **RTX 5080** (Blackwell — needs CUDA 12.8+ / a cu128 PyTorch build; older wheels will not work)

Observations are **privileged simulator state** for now; vision is a later option.

Folder layout

RL_DT/	documentation and the record of work — no RL code lives here
├ CLAUDE.md	this file — the map (keep short)
├ _docs/	everything else, see _docs/README.md
└ _hist/	/trace output: script + raw output + NOTES.md per verified result

Planned, for the Isaac Lab track only:

├ isaac/	Isaac Lab door task (env cfg, mdp terms, task registration)
└ tests/	pytest

Superseded 2026-08-12: this file used to plan an `rl_dt` package here holding a from-scratch PyTorch port (`agents/`, `models/`, `buffers/`, `envs/`, `cfgs/`, `drivers/`, `utils/`). **It never existed** — any path beginning `rl_dt` in an old note is dead. The decision instead is to add a torch agent **inside the existing toolkit** at `<PROJECT_ROOT>/SciOptControlToolkit/jlab_opt_control/`, reusing its registry, config, and driver layers. Reason and scope: `_docs/strategy.md`; recorded in `_docs/decisions.md` under 2026-08-12.

How to run

Local track — algorithm validation in Gymnasium

From the toolkit clone, not from here. Verified working 2026-08-12.

```
cd "<PROJECT_ROOT>/SciOptControlToolkit"
.venv/Scripts/python -m jlab_opt_control.drivers.run_continuous \
  --agent KerasTD3-v0 --env Pendulum-v1 --nepisodes 100 --nsteps 200 \
  --logdir ./results/Pendulum --index 1
tensorboard --logdir ./results
```

Baseline to match: KerasTD3-v0 on Pendulum-v1 crosses avg-20 of ~200 at episode 47 and ends at -163.0 in 4 min 34 s on laptop CPU. Single seed, and the driver seeds from wall clock, so it is not repeatable — see `_docs/PUBLICATION_MATERIALS.md`.

Isaac Lab track — the door task

Run through Isaac Lab's launcher, not bare python:

```
<ISAACLAB>/isaacclab.sh -p isaac/train_rl_games.py --task Isaac-UR5e-DoorOpen-v0 \
  --num_envs 256 --max_iterations 1000 --headless
<ISAACLAB>/isaacclab.sh -p isaac/play_rl_games.py --task Isaac-UR5e-DoorOpen-v0 \
  --checkpoint logs/rl_games/ur5e_door_open/<date>/nn/ur5e_door_open.pth
```

Drop `--headless` to watch in the GUI. Start at `--num_envs 1` when debugging a scene change. The reference config uses 512 envs with a 32768 minibatch — reduce both if VRAM is tight on the 5080.

Working practice — Rounds

Work is organized into **rounds**: one continuous block (a day, or several consecutive days) on a coherent goal. One round = one session = one file in `_docs/rounds/`, named `Round-YYYY-MM-DD-<slug>.md`. `_docs/WORKLOG.md` is the index; detail lives in the round file.

Review discipline — this is the default unless Dewei says otherwise:

- **One component per turn.** Don't batch several files or features into a single response.
- **Stop after each** and show the diff plus the actual test output. Wait for review before starting the next component.
- **Never report something as working without having run it.** If a test fails, show the failure.
- Log progress in the round file as you go, not at the end.
- Commit in small, reviewable units.

Start a session from the **project root** (`...\RL_Twin`), not from `RL_DT` — session history is keyed by working directory, and switching it orphans previous sessions from `/resume`.

Code conventions

- **No hyperparameters in code.** Everything lives in the toolkit's `cfigs/*.cfig` (JSON), selected by string ID — ours is `cfigs/torch_td3.cfig`. This is the one pattern worth keeping from the reference toolkit.
- **Fixed seeds, always.** The reference toolkit seeds from wall-clock time and is not reproducible; do not copy that. Seed Python, NumPy, and Torch, and log the seed.
- **Port logic, not framework.** When translating from Keras, reimplement the math in idiomatic PyTorch — no TF compatibility shims.
- **Type-hint public functions;** keep reward/observation terms as small pure functions taking `(env, params) -> torch.Tensor`.
- **Log the git SHA and full config** with every run.
- Never commit checkpoints, logs, run outputs, USD assets, or datasets — see `.gitignore`.
- Keep the training reward and the reported success metric separate, and both in the logs.

Decisions

Logged in `_docs/decisions.md`. Standing ones worth having in context: PyTorch over TensorFlow · `rl_games` PPO for the Isaac Lab track · privileged sim state for observations · add a torch agent inside the existing toolkit rather than writing a new package.

Current state

As of 2026-08-19.

- **The PyTorch TD3 port is done.** TorchTD3-v0 in `SciOptControlToolkit/jlab_opt_control/agents/torch_td3.py`, with `models/torch_models.py`, `core/torch_model_core.py` and `cfgs/torch_td3.cfg`. Commits `ee4ced6`, `61646af`, `5c928ff`, `a085ad9` on branch `develop-dw` — **1,376 insertions, 0 deletions**, so no existing toolkit behaviour changed. 82 tests pass, and the 47 on the critic and agent are mutation-tested.
- **It matches the Keras baseline on four Gymnasium environments** — ahead on two, behind on one, tied on one. Caveat that travels with it: **one seed per arm**, and the Keras arm cannot be re-seeded because it seeds from wall clock. Numbers and logdirs in `_docs/PUBLICATION_MATERIALS.md`.
- Also landed: `envs/robotics_flat.py`, opening all 217 gymnasium-robotics environments to the toolkit by flattening `Dict` observation spaces at the registry.
- Toolkit clone installed and running: Python 3.11.9, TF 2.21.0 and torch 2.13.0+cpu in one env, no CUDA on this laptop. `run_continuous.py` works and KerasTD3-v0 solves `Pendulum-v1`.
- **In flight:** a fifth environment, `AdroitHandDoor-v1`. `_docs/WORKLOG.md` carries an **estimate explicitly marked to be replaced** by the measured result — replace it, don't quote it.
- **Not started:** SAC and DDPG ports, vectorized rollout, and anything on the Isaac Lab track. The inherited door task has been read closely but never run — no RTX GPU on this machine.

Hop 2 of 5 — every number and every defect

RL_DT/_docs/PUBLICATION_MATERIALS.md - 328 lines

Scan the findings index first, then read only the entries that matter to you. The most valuable file in the repo.

PUBLICATION_MATERIALS

Things learned that could end up in a paper, a report, or a group presentation. Distinct from `WORKLOG.md` — that records *what was done*, this records *what is worth telling someone else*.

This is the canonical home for every measured number and every code defect in the project. Other files link here rather than restating. Every entry attaches its **evidence** — logdir, commit SHA, command line — because an unattributed claim cannot go in a paper.

Findings index

Scan this; read only the entries you need. Sixteen entries.

Date	The claim	
08-18	<code>TorchTD3-v0</code> reproduces the Keras baseline across four Gymnasium environments — ahead on two, behind on one, tied on one	→
08-12	Keras TD3 solves <code>Pendulum-v1</code> on stock cfg — the number the port had to match	→
08-19	Mutation testing, not a green suite, is what makes tests evidence — three injected defects, caught by 3 / 5 / 1 tests	→
08-18	Mixed-direction results are the <i>correct</i> evidence for a faithful port; the tie on the shared failure case is the strongest single piece	→
08-17	Weight initialization is a silent confound across frameworks — Glorot vs Kaiming is a factor of 1.7 at 256 wide, so it must be a named config key	→
08-18	Flattening <code>Dict</code> observation spaces at the registry rather than the driver opens 217 goal-conditioned envs — and the sorted-key layout must be asserted, not assumed	→
08-12	Only the <i>agent</i> layer needs PyTorch; the driver and buffer need work for vectorization , not framework — conflating the two is the mistake	→
08-12	TD3 is the extension point the method contribution is expressed through — every research variant overrides exactly one function, <code>train_critic</code>	→
08-10	Store <code>terminated</code> , never <code>terminated</code> or <code>truncated</code> — zeroing the bootstrap at a time limit teaches the critic that good states are worthless	→
08-12	The reference toolkit's "best model" criterion can only ever say yes: an averaging window in the wrong units, plus a relative threshold that inverts on negative rewards	→
08-12	Seeding the <i>training</i> env does not make evaluation reproducible — a 170-point evaluation spread at one fixed seed, and the evaluation number is the published one	→
08-12	The reference replay buffer's sampling cost is linear in occupancy — 82 μ s at 5k rows, 16.2 ms at 1M — so it is invisible on a smoke test and dominant later	→
08-19	A model loader that fails silently: six files matched by prefix, any miss continues with random weights, so an inference number can be a random policy's number	→
08-19	Running the reference test suite writes real training runs into <code>results/</code> , indistinguishable from experiments	→
08-10	A green <code>Pendulum-v1</code> smoke test cannot validate termination handling — its <code>terminated</code> is always <code>False</code>	→
08-10	The reference toolkit is not reproducible as published — three wall-clock seeds and an unseeded action space	→

Results

2026-08-18 — `TorchTD3-v0` reproduces the Keras baseline on four Gymnasium environments

Final avg-20 reward, one seed per arm, laptop CPU (no CUDA), stock cfgs with the 11 shared keys byte-identical between `torch_td3.cfg` and `keras_td3.cfg`. Toolkit commit `a085ad9`, branch `develop-dw`.

Env	Episodes	TorchTD3-v0	KerasTD3-v0	logdir under results/
Pendulum-v1	100	-149.29	-157.63 / -162.98	Pendulum/index010, index000, index001
Hopper-v5	300	180.12	168.29	Hopper/index010, index020
MountainCarContinuous-v0	100	-1.05	-1.01	MountainCarContinuous/index030, index031
FetchReachDense-Flat-v0	500	-1.51 (best -1.07)	-1.22	FetchReach/index040, index041

Two secondary numbers worth reporting: on `Pendulum-v1` the Keras arm crosses `avg-20 = -200` at episodes 47 and 48 while the torch arm does not until 57 — slower to take off, better at the end. On `FetchReachDense-Flat-v0` the torch arm finished the same 500 episodes in **6 m 00 s against 10 m 13 s**, 40% faster in wall clock, while converging slightly worse.

Caveats that must travel with these numbers. One seed per arm. The Keras agent seeds from `time.time_ns()`, so its arm is not reproducible even in principle; the torch agent takes a `seed cfg` key and is. Three of the four environments never set `terminated`, so only `Hopper-v5` exercises the `(1 - done)` factor in the Bellman target.

2026-08-12 — Keras TD3 baseline on Pendulum-v1 (the number the port must match)

Reference arm for the torch-vs-Keras comparison. Stock `cfgs/keras_td3.cfg`, no tuning.

avg-20 training reward, first crossing of -200	episode 47
final avg-20 (episode 100)	-163.0
best single episode	-0.9
wall clock	4 min 34 s, CPU
per-step cost once training starts	≈18 ms

Scale: random policy ≈ -1200 , do-nothing ≈ -1900 , "solved" ≤ -200 . **Caveat that must travel with this number:** it is a *single* run, and the driver seeds from `time.time_ns()` (see Failure modes), so it is not repeatable. The published comparison needs ≥ 3 runs per arm with variance. Useful shape check: our standalone PyTorch demo crossed -200 at $\approx$ episode 38 with `warmup_size 1000`, versus 47 here with `warmup_size 2500` — the same curve offset by the extra warm-up, which is what it should be. Evidence: `SciOptControlToolkit @ 48e624f`, Python 3.11.9 / TF 2.21.0 / numpy 2.4.6, HP Elite x360 G11 CPU (no CUDA). Logdir `results/Pendulum/index001_env_Pendulum-v1_agent_KerasTD3-v0_hash_48e624f_time_20260812-150620`. Command: `python -m jlab_opt_control.drivers.run_continuous --agent KerasTD3-v0 --env Pendulum-v1 --nepisodes 100 --nsteps 200 --logdir ./results/Pendulum --index 1`.

Method choices

2026-08-19 — Mutation testing is what turns a passing suite into evidence

47 unit tests on the ported critic and agent passed on the first run, which is weak information — it is equally consistent with tests that assert nothing sharp. Three defects were injected into `agents/torch_td3.py`, one at a time, and the suite re-run:

Injected defect	Tests that failed
<code>torch.minimum</code> → <code>torch.maximum</code> in the clipped double-Q target	3
<code>(1.0 - dones)</code> → <code>dones</code> in the Bellman target	5
tau swapped in <code>soft_update (mul_(tau).add_(sp, alpha=1-tau))</code>	1

The `(1 - dones)` case is the one that justifies the effort: three of the four environments used in this round never terminate early, so an inverted done flag is invisible in every reward curve except Hopper's. Reporting "N tests pass" says nothing; reporting which injected defects the suite catches says what the suite is worth.

Note on the tau result: only one test catches the swap, and that is correct rather than thin. The tau = 0.5 midpoint case is symmetric under the swap and *should* still pass; a dedicated direction test at tau = 0.25 is what detects it. A suite needs an asymmetric case, not just a midpoint case.

2026-08-17 — Weight initialization must be an explicit config key when comparing frameworks

Keras Dense initializes Glorot-uniform; PyTorch Linear initializes Kaiming-uniform. For a 256×256 hidden layer the bounds are $\sqrt{6/512} = 0.1083$ and $1/\sqrt{256} = 0.0625$ — a factor of 1.7. Left at framework defaults, any "PyTorch vs TensorFlow" learning curve is partly an *initialization* comparison, and the confound is invisible in the code because neither side names the initializer. Exposed as a cfg key `weight_init`, set to Glorot to match the reference, and pinned by a test that fails if the torch default returns. Generalizes: when porting across frameworks, every silent default that differs is a confound until it is named in config.

2026-08-18 — Flattening Dict observation spaces at the registry, not in the driver

Goal-conditioned environments (all 217 gymnasium-robotics ids: Fetch, Maze, Hand, Kitchen) publish a Dict observation space of `achieved_goal` / `desired_goal` / `observation` whose `.shape` is `None`. `run_continuous.py:191` reads `env.observation_space.shape[0]`, so every one of them raises `TypeError` before training starts. A `FlattenObservation` wrapper concatenates the sub-spaces into a single `Box` and the existing agents consume them unchanged.

Two properties worth stating in a methods section. First, `FlattenObservation` concatenates in **sorted key order**, so a Fetch observation is `achieved_goal(3) | desired_goal(3) | observation(10) = Box(16)` — the agent's input layout depends on that ordering, so it is asserted against the unwrapped env at a fixed seed rather than assumed. Second, flattening is necessary but not sufficient: `FetchPush`, `FetchSlide` and `FetchPickAndPlace` still need Hindsight Experience Replay, which the reference toolkit does not implement.

Applied as an additive registry factory rather than a driver patch, because the toolkit is upstream code — see `decisions.md`, 2026-08-17.

2026-08-12 — Only the *agent* layer needs PyTorch, not the whole toolkit

Isaac Lab's `ManagerBasedRLEnv` returns `torch.Tensor` observations already resident on the CUDA device (`[num_envs, obs_dim]`) and expects a CUDA tensor action back. A TensorFlow agent forces a GPU→CPU→GPU copy per step, which negates the massively-parallel-env design that is Isaac Lab's entire reason for existing; TF and Torch also contend for CUDA context and VRAM in a single process, and Isaac Sim containers ship torch, not TF. The registry, config, and ABC layers of `jlab_opt_control` are framework-agnostic and are reused unchanged. **Second-order point worth stating in a Methods section:** the driver and replay buffer also need rework, but for **vectorization**, not framework — `run_continuous.py` assumes a single env, a scalar reward, and one gradient step per env step. Conflating "port to torch" with "support N parallel envs" is the mistake to avoid. Detail: `strategy.md`.

2026-08-12 — TD3 is the extension point the method contribution is expressed through

In `jlab_opt_control`, every research variant is a subclass of the TD3 agent that overrides `train_critic` and changes almost nothing else: `KerasSINDyCriticTD3(KerasTD3)`, `KerasUncertaintyTD3(KerasTD3)` (147 lines total), `KerasSINDyUncertaintyTD3` (94 lines, mostly composition). So the SINDy-structured critic and the uncertainty-aware critic are not separate algorithms — they are **modifications of the Bellman-target computation inside TD3**, which is the cleanest possible framing for a Methods section: baseline TD3 and the variants differ in exactly one function. The practical consequence for the port is that a clean overridable `train_critic` is the design property that matters most, and it is why the port order is TD3 first. Evidence: `SciOptControlToolkit` @ 48e624f, `jlab_opt_control/agents/keras_sindy_critic_td3.py:54`, `keras_uncertainty_td3.py:46`, `keras_sindy_uncertainty_td3.py`.

2026-08-10 — Store terminated, never terminated or truncated

The Bellman target's `(1 - done)` factor must zero the bootstrap only on genuine termination. Zeroing it at a time limit teaches the critic that good states are worthless. The reference toolkit gets this right. Worth stating explicitly because it is invisible in results — see Failure modes.

Failure modes

2026-08-19 — Running the reference test suite writes training runs into the results tree

`pytest` `utests/` produced **11 real training-run directories** inside `results/`, at commit `a085ad9`, timestamps 2026-08-18 23:32–23:56. `utests/test_baselines.py` and `utests/test_agents.py` train actual agents, and the driver resolves its logdir relative to the working directory, so the runs land wherever `pytest` was invoked. In a TensorBoard listing they are indistinguishable from experiments — `index000_env_Pendulum-v1_agent_KerasTD3-v0_hash_...` looks exactly like a real result, and two of the eleven were `KerasDDPG-v0` and `KerasSINDyCriticTD3-v0` runs nobody launched on purpose. Anyone reporting numbers from a shared results tree can pick up a test artefact by mistake. Deleted; new tests in this project use `tempfile.mkdtemp()` as logdir for exactly this reason.

Secondary: `pytest` `utests/` exceeds ten minutes as a whole, `test_baselines.py` alone taking over three, so it is not usable as a pre-commit check in its current form.

2026-08-19 — A model loader that fails silently and reports a random policy's numbers

`TorchTD3.load()` — and the Keras agent it was ported from — wraps the whole load in a `try/except` that logs an error and **continues with randomly initialized weights**. TD3 needs six files matched by name prefix; if the path is wrong, or one file is missing, or a shape mismatches, training or `--inference True` proceeds against an untrained network. The only signal is one `ERROR` line in a log that also carries routine `ERROR:CfgLogger` noise from missing optional `cfg` keys, so it does not stand out. Consequence for a results table: an inference number can be a random policy's number, with nothing in the output to say so. Now covered by two negative tests (a five-of-six-file directory and a missing directory) that assert the error is actually emitted — but the silent fallback itself is upstream behaviour and unchanged.

2026-08-10 — A green `Pendulum-v1` smoke test does not validate termination handling

Pendulum's `terminated` is *always* `False` — every episode ends by truncation at 200 steps. So the `terminated-vs-truncated` bug above cannot be detected on Pendulum at all. Any task with real termination (the door task) is the first place it shows up, and it shows up only as "learning is quietly worse." Implication for the paper's reproducibility section: state which envs exercise which code paths.

2026-08-12 — Seeding the training env does not make *evaluation* reproducible

A distinct trap from the wall-clock-seeding one below, and easier to fall into because the training side looks correct. In our own `learn/td3_pendulum_demo.py` — a file whose comments explicitly criticise the reference toolkit's seeding — training was bit-reproducible under a fixed seed while the reported evaluation return varied by **170 points** across four runs of the same seed (−1621.4 / −1550.4 / −1525.9 / −1449.8, identical training returns of −1508.2 throughout). Cause: the separate `eval_env` had its *action space* seeded but its *state* RNG left to OS entropy, because `evaluate()` calls `reset()` with no seed.

Why it belongs in a paper's reproducibility section: **the evaluation number is the one that gets published**, and it is the one an unseeded eval env silently randomises. On Pendulum, whose start angle is uniform over the circle, a 3-episode greedy evaluation carries enough start-state variance to swamp a real difference between two methods. Any A/B claim built on few-episode evaluations needs the eval env seeded independently of the training env, or enough eval episodes to average the start-state distribution out — and the paper should say which.

The same structural trap exists in the reference toolkit and is worse there: `drivers/run_continuous.py:258` runs its inference episodes on the **same env object** used for training, so evaluation consumes and advances the training env's RNG stream — evaluation and training are not even statistically independent. Evidence: `RL_DT@develop learn/td3_pendulum_demo.py:241, :266-268`; runs recorded in `rounds/Round-2026-08-12-torch-agent-in-toolkit.md`; `SciOptControlToolkit@48e624f jlab_opt_control/drivers/run_continuous.py:258`.

2026-08-12 — The reference replay buffer's sampling cost is linear in buffer occupancy

`jl原因ab_opt_control/buffers/er.py, ER.sample()`, is `np.random.choice(max_index, size=nsamples, replace=False)` where `max_index = min(pointer, capacity)`. With `replace=False` and no probability vector, NumPy materialises a **full permutation of `max_index`** to draw `nsamples` indices from. Measured (HP Elite x360 G11, CPU, numpy 2.4.6, Python 3.11.9; 256 indices, best-of-5 of 10 calls, via `uv run --with numpy`):

rows in buffer	replace=False	replace=True
5,000	82 μs	10.4 μs
50,000	810 μs	10.1 μs
100,000	1.64 ms	10.2 μs
1,000,000	16.2 ms	9.3 μs

Linear in occupancy; the with-replacement path is flat. **Why it matters for the paper:** the cost is driven by how *full* the buffer is, not by the configured capacity, so it is invisible on the standard `Pendulum-v1` smoke test (a few thousand rows → ~80 μs, negligible beside the gradient step) and becomes the dominant per-step cost once the buffer fills — at 1M rows, index selection costs more than the gradient step it feeds. Any sample-efficiency-vs-wall-clock comparison involving this buffer must state the buffer occupancy, or the wall-clock axis is measuring the sampler rather than the algorithm. The fix is `replace=True` (what every reference TD3 implementation does); the only semantic loss is the guarantee that a batch holds no duplicate transition, which at batch 256 out of ≥2500 rows is statistically irrelevant. Evidence: `SciOptControlToolkit @ 48e624f, jlab_opt_control/buffers/er.py:69, cfgs/er.cfg (buffer_capacity: 1000000)`. Not fixed as of this entry — deferred, see `strategy.md` Part 3.

2026-08-12 — A "best model" criterion that can only ever say yes

Observed on the first real run through the reference driver: all 10 inference checkpoints were saved as improvements, including one whose evaluation return was 98 points *worse* than an earlier one. Two independent defects in `drivers/run_continuous.py` combine:

1. **Averaging window vs. sampling interval.** :264 takes `np.mean(inference_reward_list[-nepisode_avg:])` with `nepisode_avg = 20`, but the list is appended only every `inference_interval = 10` episodes. In a 100-episode run the list has 10 entries, so the "recent average" spans the whole run including the random-policy start. It therefore rises monotonically by dilution — $-845 \rightarrow -743 \rightarrow -678 \rightarrow -603 \rightarrow -565$ over episodes 50–90 — while the underlying evaluations bounced $-117 / -129 / -223 / -4 / -227$.
2. **A relative improvement threshold applied to a negative quantity.** :266 is `avg > best * (1 + threshold)`. For `best < 0` this *lowers* the bar by `threshold`, so the 5 % margin intended to demand real improvement instead admits results up to 5 % worse. It behaves as designed only for positive-reward tasks — and both Pendulum and accelerator tuning-error rewards are negative.

Why it belongs in a paper's reproducibility section: this is the class of bug that makes reported numbers *better* than the method deserves, because "best checkpoint" evaluation silently degenerates into "last checkpoint," and any relative-improvement gate on a negative objective inverts. It is worth stating as a general rule: **relative thresholds must be written sign-safe** (`best + t*abs(best)`), and any "average of the last N" must be in the same units as the thing being counted. Neither error is visible in a training curve; both are visible only by counting how many checkpoints were accepted. Evidence: SciOptControlToolkit @ 48e624f, `jlab_opt_control/drivers/run_continuous.py:264, :266, :281, :385, :387, :391`; `run logdir results/Pendulum/index001_env_Pendulum-v1_agent_KerasTD3-v0_hash_48e624f_time_20260812-150620 (models/epoch_00000_000 ... epoch_00090_006, 10 acceptances out of 10 opportunities)`. Not fixed — shared driver code, raised as Q9.

2026-08-10 — The reference toolkit is not reproducible as published

`run_continuous.py` seeds from `time.time_ns()`, and `keras_td3.initialize_new_models()` re-seeds from wall-clock again between critic constructions. The action space is never seeded, so the warm-up (thousands of random transitions) varies run to run even if everything else is fixed. Any comparison against it needs multiple seeds and reported variance, not single runs.

Comparisons

2026-08-18 — Mixed-direction results are the correct evidence for a faithful port

Across four environments TorchTD3-v0 is ahead on two (Pendulum-v1, Hopper-v5), behind on one (FetchReachDense-Flat-v0) and tied on one (MountainCarContinuous-v0). Numbers in **Results, 2026-08-18**.

That pattern is the claim, not a weakness in it. A port that swept all four would more likely indicate a changed hyperparameter or a different effective exploration rate than a better implementation — the two agents share 11 byte-identical `cfg` keys precisely so that neither should dominate.

The strongest single piece of evidence is the **tie on the failure case**. On MountainCarContinuous-v0 both agents drop from roughly -33 to -1.0 by episode 3 and reach the goal in **0 of 100 episodes**, converging on "spend no energy". This is TD3's canonical hard-exploration failure: Gaussian noise at 10% of the action range never carries the car to the flag. Reproducing a known failure mode identically, without having aimed at it, is harder to achieve by accident than matching a success.

Remaining weakness, stated for a reviewer: one seed per arm, and the Keras arm cannot be re-seeded because that agent seeds from wall clock. ≥ 3 seeds per arm is still outstanding.

Figures

(none yet)

Related work

(none yet — start with the SINDy / physics-informed-critic papers behind `keras_sindy_critic_td3.py`, and the uncertainty-aware TD3 variants)

Buckets, for filing: Results = measured numbers with conditions attached · Method choices = a design decision plus the reason · Failure modes = things that silently degrade learning · Comparisons = baseline vs variant, with caveats · Figures = what plot from which logdir · Related work = papers read, and how they differ from what we do. A one-line stub with a date beats nothing, and every entry adds a row to the index above.

Hop 3 of 5 — what was done, round by round

RL_DT/_docs/WORKLOG.md - 154 lines

The three round files it links to are NOT in this packet - open them in the repo only when you need provenance for a specific claim.

WORKLOG

What was done, round by round — by Dewei or by the coding agent. Newest round at the top.

One line per round in the index; the narrative detail lives in the round file under `rounds/`. Anything worth *publishing* (a result, a measured number, a lesson, a figure) goes to `PUBLICATION_MATERIALS.md` instead of here — this file is the record of work, that one is the record of findings. Keep each round entry to about six bullets.

Index

Round	Goal	Status
2026-08-19 (docs only, no round file)	Reorganize <code>_docs/</code> : 36 files → 16, four learning tracks, one home per fact	done — see below
Round-2026-08-17	Install the agent commands, reorganize the docs, then write the PyTorch TD3 agent	done — <code>TorchTD3-v0</code> written, 82 tests, matched against Keras on 4 environments; a 5th in flight
Round-2026-08-12	Add a PyTorch TD3 agent to <code>jlab_opt_control/agents</code> , wire it to a Gymnasium env, understand the kit	done — env installed, Keras baseline solved Pendulum; agent landed in the 08-17 round
Round-2026-08-10	Port the RL algorithms to PyTorch, validate in Gymnasium, prepare for Isaac container integration	superseded — see 08-12

2026-08-19 — Documentation reorganization

No code touched, no results generated. `_docs/` had grown to 36 files, half of them pre-code material filed as "closed" while holding the project's only Isaac primer and its best toolkit walkthrough; the same facts were restated three or four times; and open questions were scattered across six files addressed to one person.

- **Four learning tracks in `learn/`**, one file each — RL and TD3, the toolkit, Isaac and `rl_games`, the door task — plus the one runnable file (`td3_pendulum_demo.py`) and a `learn/README.md` that carries the prerequisite chain and a printable five-file review list.
- **`team-discussion.md` replaces six question sources.** Present · ask · propose: how the work is described, the open questions grouped by *who can answer them*, and three upstream patches written out rather than applied.
- **`strategy-torch-and-isaac.md` → `strategy.md`**, re-cast from proposal to status: PORT done, REUSE held, DEFER still deferred.
- **Single-sourcing rule adopted:** a measured number or a code defect lives in `PUBLICATION_MATERIALS.md` and nowhere else. Duplicate tables and restated defect lists collapsed to links. `PUBLICATION_MATERIALS.md` gained a findings index so it can be scanned rather than read.
- **`CLAUDE.md` § Current state corrected** — it still said "No agent code written yet", which every other file contradicted — and the two completed short-term roadmap items ticked.

- **27 files deleted in a second commit** (3123e27), after the salvage commit (e752e85, zero deletions), so `git log --follow` recovers any of them: `archive/` (17), `meetings/` (2), `learning/` (4 — the fifth, `td3_pendulum_demo.py`, moved to `learn/`), `setup/` (1), `rounds/README.md`, `questions-for-malachi.md`, `strategy-torch-and-isaac.md`. Result: **36 files → 16** under `_docs`, 3,815 lines including `CLAUDE.md` and `_hist/README.md`; 134 relative links and all 16 findings-index anchors resolve. Mapping:

Deleted	Where it went
<code>archive/inherited-2026-07-24/01_Project_Overview.md</code> , <code>04_Paper_Skeleton_Thoughts.md</code> , <code>Attachment_RL_Understanding.md</code>	<code>learn/4-door-task.md</code>
<code>archive/inherited-2026-07-24/02_DigitalTwin_Detailed_Guide.md</code> , <code>02_DigitalTwin_Brief.md</code>	<code>learn/3-isaac-and-rl-games.md</code> (with a correction header — its "there is no code in this package" claim was wrong)
<code>archive/inherited-2026-07-24/03_RL_Toolkit_Detailed_Guide.md</code> , <code>03_RL_Toolkit_Brief.md</code>	<code>learn/2-toolkit.md</code> (§ 9 and § 10 only; its smoke test and "quirks" are superseded by <code>CLAUDE.md</code> and the measured failure modes)
<code>archive/inherited-2026-07-24/01_Questions_for_ProjectManager.md</code> , <code>02_Questions_for_DT_Developer.md</code> , <code>03_Questions_for_Malachi.md</code> , <code>Attachment_Questions_for_Malachi.md</code>	<code>team-discussion.md</code> Part 2
<code>archive/inherited-2026-07-24/00_START_HERE.md</code>	superseded by <code>README.md</code> + <code>learn/README.md</code>
<code>archive/brainstorm-2026-06/SciOptControlToolkit_Walkthrough.md</code>	<code>learn/2-toolkit.md</code> (a PDF copy also survives at <code>_reference/SciOptControlToolkit_Walkthrough.pdf</code>)
<code>archive/brainstorm-2026-06/rl_development_plan_*.md</code> (504 lines), <code>robotics_rl_digital_twin_brainstorm.md</code> (489)	nothing salvaged — pre-code LLM brainstorm, superseded by <code>CLAUDE.md</code> 's roadmap, <code>strategy.md</code> and <code>learn/4-door-task.md</code> . <code>_reference/LLM_brainstorm/</code> no longer exists, so git history is the only remaining copy
<code>archive/superseded/</code> (2)	dead — the retired <code>_notes/README</code> and the retired <code>worklog</code>
<code>meetings/README.md</code> , <code>meetings/upcoming-malachi.md</code>	<code>team-discussion.md</code> Parts 1 and 2
<code>setup/aws-bedrock-vscode-setup.md</code>	<code>environment.md</code> , final section
<code>learning/td3_pendulum_notes.md</code> , <code>gymnasium-basics.md</code> , <code>pytorch-vs-tensorflow.md</code>	<code>learn/1-rl-and-td3.md</code>
<code>learning/td3_pendulum_demo.py</code>	<code>learn/td3_pendulum_demo.py</code> (moved, three stale paths fixed)
<code>learning/README.md</code>	<code>learn/README.md</code> — the old one indexed 2 of its 4 files
<code>rounds/README.md</code>	naming convention → <code>README.md</code> ; its kickoff prompt is redundant with <code>~/.claude/commands/round-start.md</code>
<code>questions-for-malachi.md</code>	<code>team-discussion.md</code> Part 2, with Q3/Q4/Q6 moved to "already answered by our own work"

Round-2026-08-17 — Harness, documentation layout, then the torch agent

Detail: rounds/Round-2026-08-17-torch-td3.md.

- Installed the five slash commands to `~/ .claude/commands/` (`round-start`, `wrap`, `trace`, `handoff`, `slides`), verified byte-identical to the source. Reorganized all documentation into `_docs/` grouped by lifetime — `/round-start` and `/wrap` look for `_docs/WORKLOG.md` and would otherwise have found the stale `_notes/worklog.md`.
- **Wrote TorchTD3-v0 and its two models.** Commits `ee4ced6`, `61646af` and `a085ad9` on toolkit branch `develop-dw`: **1,376 insertions, zero deletions** — the only edits to pre-existing files are added lines in three `__init__.py` files and `.gitignore`.
- **All hyperparameters in `cfgs/torch_td3.cfg`, none in code.** 17 keys against Keras's 11; the 11 shared keys are byte-identical, which is what makes the comparison controlled. Four of the extra six promote hardcoded Keras literals at the same numbers; `seed` and `device` have no Keras counterpart.
- **Matched the Keras baseline on four environments** — ahead on two, behind on one, tied on the fourth. Numbers, logdirs and caveats → `PUBLICATION_MATERIALS.md`.
- **Opened the 217 gymnasium-robotics environments to the whole toolkit** with `envs/robotics_flat.py`, an additive registry factory that wraps `Dict` observation spaces in `FlattenObservation`. Built additively rather than as a driver patch, so the 3-line driver fix became an upstream proposal — `team-discussion.md` Part 3.
- **82 unit tests, 719 lines of test code**, then verified they fail on a real bug by injecting three defects and confirming each was caught. Both → `PUBLICATION_MATERIALS`.
- **Carried over:** the fifth environment, `AdroitHandDoor-v1`, was in flight at wrap time. A 1000-episode run per arm launched 2026-08-19 00:44; at episode 145 the Torch arm's avg-20 had moved `-43.78` → `-36.77`. **Estimate, to be replaced by the measured result:** both arms plateau around `-25` to `-32` avg-20 and **do not open the door** — 200k steps is one to two orders of magnitude below published Adroit results. Whoever reads this next replaces the estimate.

Round-2026-08-12 — PyTorch agent inside the toolkit

Detail: rounds/Round-2026-08-12-torch-agent-in-toolkit.md. Work happens in **two repos**: toolkit code in `<PROJECT_ROOT>/SciOptControlToolkit` (branch `develop-dw`, HEAD `48e624f` at the time), notes and logs here.

- Read the toolkit end to end and derived the agent interface **from the driver and the tests**, not from the ABC — `core/agent_core.py` omits `memory` and `buffer`, both of which the driver requires.
- **Installed the toolkit environment and ran the kit for the first time.** `uv venv --python 3.11`; the whole declared dependency set resolves on **Python 3.11.9**, TF 2.21.0 and torch 2.13.0+cpu in one env, so `env.yaml`'s `python=3.13` pin is not a hard floor. `torch>=2.10` was already declared and imported nowhere, so the port added no dependency. Machine correction: this is the laptop (`WF10878`, Intel Arc, **no CUDA**), not the RTX 5080 workstation.
- **Keras baseline established** — `KerasTD3-v0` solves `Pendulum-v1` on stock `cfg`. → `PUBLICATION_MATERIALS`.
- Wrote the strategy note (`strategy.md`): the three-tier PORT / REUSE / DEFER scope, the argument that driver and buffer work is about **vectorization not framework**, and architectures A/B/C with **B** recommended. Driving finding: every research variant subclasses `KerasTD3` and overrides `train_critic`, so a clean overridable seam is the port's most important property.
- Two things the first real run taught that reading could not: the `warmup_size` gate is visible in the wall clock, and the "best model" checkpoint criterion never rejects anything. Also measured a real performance bug in `buffers/er.py`. All three → `PUBLICATION_MATERIALS`, failure modes.

- Audited notebooks/: all four analyse a *trained* checkpoint, none trains anything, three need pysindy (declared nowhere), and ExaminePACESPolicy.ipynb hardcodes another developer's Mac path.

Round-2026-08-10 — PyTorch port (learning pass)

Detail: rounds/Round-2026-08-10-pytorch-port.md.

- Read CLAUDE.md, environment.md, decisions.md, then the reference toolkit end to end.
- Environment check: **no NVIDIA GPU on the laptop**. The RTX 5080 / cu128 requirement describes the PNNL workstation only. Not a blocker for Pendulum-scale TD3. Target Python **3.11**.
- Learning notes written, now merged into learn/1-rl-and-td3.md with the standalone demo at learn/td3_pendulum_demo.py.
- No production code written. **Superseded**: the plan was a from-scratch rl_dt/ package; the 08-12 decision is to add a torch agent *inside* the existing toolkit instead.

Before rounds (setup phase)

- **2026-08-12** — answered "does the whole kit need re-drafting in PyTorch?" (no); cloned JeffersonLab/SciOptControlToolkit to <PROJECT_ROOT>/SciOptControlToolkit on branch develop-dw as a live working copy; adopted WORKLOG.md + PUBLICATION_MATERIALS.md and retired _notes/worklog.md; documented FAI/Bedrock billing for the VS Code extension (now the last section of environment.md).
- **2026-08-04** — Z:\RL_Twin no longer resolves; the canonical path is the OneDrive folder and Z:\ was purged from CLAUDE.md. Project documentation moved inside the repo so it travels with git clone; _reference/ (~2 GB) stays out.
- **2026-08-03** — read all of _docs/, skimmed both reference codebases, and found that the DT package is **not asset-only**: _reference/digital_twin_models-main/RL_CloudTesting/ holds a working Isaac Lab ManagerBasedRLEnv door task with rl_games train/play scripts. Gaps in it: no force/torque penalty, no domain randomization, heavy script-guided rewards — all three are now roadmap items. Wrote CLAUDE.md and set up the repo.

Hop 4 of 5 — scope and the open architecture choice

RL_DT/_docs/strategy.md - 293 lines

The review list calls for Parts 2 and 5. Part 4's A/B/C architecture choice is still open.

Strategy — how much of SciOptControlToolkit becomes PyTorch, and how it meets Isaac Lab

Written 2026-08-12 as a proposal, opening Round-2026-08-12. **Re-cast 2026-08-19 as a status note:** the scope in Part 2 has been executed, so each tier now records what closed it and with what evidence. Part 4's architecture choice is **still open** and is the top item in `team-discussion.md`.

Purpose: settle the scoping question ("does the whole toolkit need re-drafting in PyTorch?") with evidence, and lay out the three candidate architectures for the Isaac Lab track so they are discussed rather than decided unilaterally.

Code citations are to `<PROJECT_ROOT>/SciOptControlToolkit`, branch `develop-dw`, at commit **48e624f** when the note was written. Line numbers not; file and symbol names do not. Every measured number and every defect referenced here lives in `PUBLICATION_MATERIALS.md`, not here.

The question, and the answer in one line

No. Exactly one layer of the toolkit is framework-bound in a way that matters — the agent and its networks. Everything else is either framework-agnostic already, or needs work for a reason that has nothing to do with TensorFlow vs PyTorch.

Conflating those two reasons is the mistake this note exists to prevent.

Part 1 — Why the agent layer has to be PyTorch

Not an aesthetic preference. Three concrete reasons, in descending order of how hard they are to work around.

1. Isaac Lab's data never leaves the GPU, and TF cannot receive it there. `ManagerBasedRLEnv.step()` returns `torch.Tensor` observations of shape `[num_envs, obs_dim]` already resident on the CUDA device, and expects a CUDA tensor action back. A TensorFlow agent in that loop forces, **every environment step**:

```
GPU (torch obs) → CPU (numpy) → GPU (tf tensor) → [network] → CPU (numpy) → GPU (torch action)
```

Four device transfers where the correct number is zero. Isaac Lab's whole design premise is that stepping 256–4096 environments is cheap *because nothing synchronises with the host*. This is not a constant-factor slowdown you can accept: the transfers serialise against the physics kernels, so the parallelism itself stops paying.

2. Two frameworks in one process contend for the same GPU. TF and Torch each create a CUDA context and each have their own allocator; TF reserves nearly all VRAM at first use. Isaac Sim is also in that process and is itself VRAM-hungry. Three allocators, one 5080. Solvable with `TF_FORCE_GPU_ALLOW_GROWTH` and careful ordering, but a permanent source of environment-specific failure for no gain.

3. The container ships torch, not TF. Isaac Sim's official containers pin a PyTorch build matched to the CUDA version; TF is absent, and making a TF wheel agree with CUDA 12.8+ / Blackwell is an install problem we would own forever. `requirements.txt:9` already declares `torch>=2.10` and nothing imports it, so **the port added no new dependency** — it started using one already sanctioned.

Consequence: the port is bounded by "what touches a neural network," not by "what is in the repo."

Part 2 — The three-tier scope, and what closed each tier

Tier 1 — PORT · done

Closed by commits `ee4ced6`, `61646af`, `5c928ff` on `develop-dw` (plus `a085ad9` for the env adapter), **1,376 insertions and zero deletions**, and 82 passing tests of which the 47 on the critic and agent were mutation-tested. → `PUBLICATION_MATERIALS.md`.

Keras file	Why it was framework-bound	Size	Ported to
<code>agents/keras_td3.py</code>	<code>tf.GradientTape</code> , <code>@tf.function</code> , Keras Adam, <code>get_weights/set_weights</code> surgery in <code>soft_update</code>	429 lines, ~250 substantive	<code>agents/torch_td3.py</code> → <code>TorchTD3-v0</code>
<code>models/actor_fcnn.py</code>	<code>layers.Dense</code> , <code>tf.constant</code> for <code>action_scale/action_bias</code> , Keras <code>call()</code>	72	<code>models/torch_models.py</code> → <code>ActorFCNNTorch</code>
<code>models/critic_fcnn.py</code>	same	68	same file → <code>CriticFCNNTorch</code>
<code>core/model_core.py</code>	<code>class Model(tf.keras.Model)</code> — the model base class <i>is</i> a Keras model	18	<code>core/torch_model_core.py</code>

~410 lines, of which maybe 250 were real logic — the estimate held. `core/model_core.py` was the surprise: unlike `core/agent_core.py` (a pure ABC with no imports), the *model* base class inherits from `tf.keras.Model`, so the torch networks cannot share a base class with the Keras ones. They subclass `torch.nn.Module` and satisfy the same duck-typed surface — `save_cfg()`, `action_scale`, `action_bias` — which is all the agent uses.

One coupling still open. `agents/__init__.py:30-36` imports all seven Keras agents at module import time and `drivers/run_continuous.py:39` imports `tensorflow` at module level, so **TensorFlow must be installed for the torch agent to run at all**. Fine on a workstation, fatal in an Isaac container. Fixing it means guarded/lazy imports in shared kit code — a conversation, not a unilateral edit, and it gates Isaac regardless of which architecture is chosen. Not in Part 3 of `team-discussion.md` only because no patch is drafted yet.

Tier 2 — REUSE UNCHANGED · held

Reused exactly as written; nothing in this tier was edited.

File	What it does	Why it is framework-blind
<code>agents/</code> , <code>models/</code> , <code>buffers/registration.py</code>	string-ID → class registry, mirroring <code>gym.make</code>	pure <code>importlib</code> ; never imports a framework
<code>core/agent_core.py</code>	the Agent ABC	38 lines, abstract methods only, no imports
<code>utils/cfg_utils.py</code>	<code>cfg_get(dict, key, default)</code>	13 lines of dict access
<code>utils/git_utils.py</code>	git SHA for the logdir name	subprocess
<code>cfgs/*.cfg</code>	JSON hyperparameters	data

Worth reusing on its own merits, not just to save effort: the registry + JSON-cfg convention is the one pattern CLAUDE.md committed to keeping, and it is what makes `--agent TorchTD3-v0` swap cleanly against `--agent KerasTD3-v0` in one command line. Reimplementing it would have *removed* the ability to A/B the two agents — which is exactly how the four-environment comparison was produced.

The caveat that turned out to matter most. `core/agent_core.py` declares only `soft_update / train / action / load / save / save_cfg`. But `drivers/run_continuous.py` also calls `agent.memory(...)` (:347) and `agent.buffer.save(...)` (:279, :299), and `utests/test_agents.py` reaches into `agent.warmup_size`, `agent.batch_size`, `agent.buffer.size()`, `.pointer`, `.states`. **The real interface is wider than the declared one** — satisfying the ABC is necessary but not sufficient. The port derived its contract from the driver and the tests instead, and 82 tests now pin it.

Tier 3 — DEFER, for vectorization not framework · still deferred

`drivers/run_continuous.py` and `buffers/er.py` both need rework before Isaac. **Neither needs it because of TensorFlow.** Detail in Part 3. The obligation was to not design anything that blocks it, and that obligation was met (see § What this round must not do, below).

Part 3 — What "vectorization, not framework" means concretely

The part worth being precise about, because "the driver needs porting too" is the plausible-sounding wrong conclusion.

The driver assumes exactly one environment

`run_episode` (`run_continuous.py`:301) hard-codes single-env shapes in its own assertions: a NumPy array of shape `(num_states,)` (:326), a scalar Python float reward (:341), and a `while not done` loop driven by one boolean. Under Isaac Lab every one is wrong in the same way — the leading `num_envs` axis is missing. `state` is `[N, obs_dim]`, `reward` is `[N]`, `done` is `[N]`, and episodes end at different times in different environments, so the loop cannot be `while not done` at all.

And the learning cadence:

```
if train:
    agent.memory((state, action, reward, next_state, terminate)) # :347 one transition
    agent.train() # :348 one gradient step
```

One transition stored and one gradient step per env step. With `num_envs=256` a single `step()` yields **256 transitions**; doing 256 separate `memory()` calls plus 256 gradient steps is both wrong (the standard is a tunable updates-to-data ratio, not 1:1) and pathologically slow. Sorting that out is a decision about *update ratio and batching* — a reinforcement-learning question. Rewriting those three assertions in torch instead of numpy would change nothing about it.

The replay buffer: NumPy is fine, but this NumPy is not

`buffers/er.py` stores plain `np.zeros((capacity, dim))`. A NumPy buffer works perfectly well with a torch agent — you convert a `batch_size × dim` slice per gradient step, a trivial copy. So **"the buffer is NumPy" is not itself a reason to touch it**. Two things are:

1. **Insertion is scalar.** `record()` (:50) writes one row per call. Isaac hands back `N` rows at once, already on the GPU; pulling them to host to store in NumPy reintroduces exactly the transfer Part 1 was about. A device-resident tensor buffer with batched inserts is what Isaac wants.

2. **Sampling is $O(\text{buffer fill level})$, not $O(\text{batch size})$** — `sample() (:69)` uses `np.random.choice(max_index, size=nsamples, replace=False)`, which materialises a full permutation of `max_index`. Measured, linear in occupancy, and invisible on a `Pendulum-v1` smoke test precisely because the cost tracks how *full* the buffer is rather than its configured capacity. The table, the conditions and the one-line fix are in `PUBLICATION_MATERIALS.md` § Failure modes.

Both are shape-and-device problems. Neither is a TensorFlow problem. Neither was fixed — this is someone else's repo, and the torch agent inherits whatever the buffer does. Raised as a proposal in `team-discussion.md` Part 3.

What this round must not do — and did not

Nothing in `TorchTD3` may assume a leading batch axis of exactly 1: `action()` takes `[obs_dim]` today but is written so `[N, obs_dim]` is a one-line change, and `train()` does not tie "one call" to "one transition arrived." That was the whole forward-compatibility obligation, and it is satisfied. Everything else stays deferred.

Part 4 — The three candidate architectures · open

The real question behind the port is what role this toolkit plays on the Isaac Lab track, given that `decisions.md` (2026-08-03) already commits to **rl_games PPO** there. Three coherent answers.

A — the toolkit replaces rl_games. `jlab_opt_control` becomes the training stack for the door task: `TorchTD3` consumes `ManagerBasedRLEnv` directly, the driver is rewritten for vectorized envs, the buffer becomes a device-resident tensor ring, PPO is eventually added. *For*: one stack for everything, so a research result is immediately a result on the real task. *Against*: we would rebuild, unfunded, what `rl_games` already does well — vectorized rollout, mixed precision, running observation statistics, multi-GPU, checkpoint/resume, LR scheduling, and the thousand numerical details that decide whether PPO trains. It also reverses a recent decision and puts the toolkit on the door task's critical path. *Cost*: weeks, ongoing.

B — rl_games is the baseline; the toolkit carries the research contribution. `rl_games` PPO trains the door task and produces the baseline numbers unchanged. `jlab_opt_control` with `TorchTD3` is the vehicle for the *method* contribution — the SINDy-structured and uncertainty-aware critics — validated on Gymnasium first, then run on the door task as the comparison arm. *For*: it matches what the code already says the research *is* (see Part 5). It preserves the `rl_games` decision, keeps the toolkit off the critical path, and gives the clean two-arm comparison reviewers want. *Against*: two stacks and two sets of results to keep honest; an off-policy-vs-on-policy comparison needs its units stated and its caveat repeated every time. *Cost*: moderate — this round plus vectorization later, and no rewriting of things that work.

C — the toolkit stays a low-parallelism research bed. Never pointed at Isaac Lab. It stays where it is strong: single- or few-env problems where sample efficiency matters and throughput does not. The door task is a separate `rl_games` effort that borrows *findings*, not code. *For*: cheapest by a wide margin, and honest about the toolkit's design centre — the vectorization work in Part 3 never happens. *Against*: the method contribution is then never demonstrated on the robot task, which is the project's actual subject. "Does the SINDy critic help on the door?" — "we don't know." *Cost*: small.

	A — replaces rl_games	B — baseline + research arm	C — separate research bed
rl_games decision	reversed	preserved	preserved
Vectorization work needed	full	eventually, for the comparison arm	never
Method result shown on the door task	yes	yes	no
Toolkit on the door task's critical path	yes	no	no
Duplicate stacks to maintain	no	yes	yes
Effort	weeks+, ongoing	moderate	small
Main risk	we rebuild rl_games badly	comparison framing	contribution never lands on the real task

Part 5 — Recommendation: B

Three reasons, in order of weight.

1. **The code already votes for it.** Every research variant subclasses the TD3 agent and overrides `train_critic`, changing almost nothing else — TD3 is not one algorithm among several here, it is the base class the contribution is expressed as an override of. B is the architecture that pattern implies. A treats the toolkit as infrastructure it was not built to be; C declines to use the extension point on the task the project is about. Evidence and file citations: `PUBLICATION_MATERIALS.md` § Method choices.
2. **It is the only option that produces the paper's central figure.** The claim "a physics-informed / uncertainty-aware critic helps on a contact-rich lab-automation task" requires running the method on that task next to a credible baseline. C cannot produce it; A produces it against a baseline we built ourselves, which is much weaker evidence than one everyone already trusts.
3. **It fails cheaply.** B's expensive part — vectorized rollout plus device-resident buffer — is *deferred and optional*. If the method shows nothing on Gymnasium we stop before paying for it, and the door task is unaffected because `rl_games` was never displaced. A commits the spend up front.

What B commits us to, in priority order:

- TorchTD3 with `train_critic` as a small, clearly-documented, overridable method — the seam the variants hang off. **Delivered:** it is the property the port was designed around.
- Agent code with no leading-batch-axis-of-1 assumption (Part 3). **Delivered.**
- Reporting the TD3-vs-PPO comparison in sample-efficiency terms with the on-/off-policy caveat attached every time, not in wall-clock alone. **Outstanding.**
- Accepting two stacks, and therefore two seeding and logging conventions that must both be reproducible. **Outstanding** — the torch side is reproducible via the `seed cfg` key; the Keras side cannot be, and the `rl_games` side pins `seed: 42`.

What would change the recommendation: if `rl_games` cannot handle the door task's force/torque observations or termination structure without modification, A gets stronger — at that point we are modifying a training stack either way, and modifying our own is preferable to patching someone else's. Worth checking early, and it is now checkable: the task's `TerminationsCfg` and the absence of any force/torque term are read out in `learn/3-isaac-and-r1-`

games.md. Conversely, if the SINDy critic needs system identification on *single* long trajectories rather than batched short ones, C gets stronger.

Part 6 — What this note does not decide

Carried into team-discussion.md:

- **The A/B/C choice itself** — still a recommendation to discuss, not a decision taken. It is question B1 there.
- **What the ported TD3 is ultimately for** — the same question from the other side (B2).
- Whether memory and buffer belong in the Agent ABC (Part 2 caveat) — moot in practice now, since 82 tests pin the real interface.
- Whether the toolkit's setup.py (packages=['jlab_opt_control'], subpackages unlisted) is meant to be installed non-editable.

Settled since: env.yaml's python=3.13 is not a hard floor — the whole dependency set resolves on 3.11.9 (environment.md). Shared vs separate critic optimizers, deterministic seeding, and torch.utils.tensorboard.SummaryWriter over tf.summary were all decided during the port and are recorded in decisions.md.

See also

- rounds/Round-2026-08-12-torch-agent-in-toolkit.md and rounds/Round-2026-08-17-torch-td3.md — the work itself
- decisions.md — the rl_games commitment this note reasons against
- learn/1-rl-and-td3.md — the idiom-level translation notes for Tier 1
- PUBLICATION_MATERIALS.md — the Methods-section version of Parts 1 and 3

Hop 5 of 5 — present, ask, propose

RL_DT/_docs/team-discussion.md - 233 lines

The only file with an action attached. Its defect table deliberately repeats hop 2 - it is a handout for someone who will not follow links.

Team discussion — present · ask · propose

A handout, not a private note. Part 1 is what I say about the work, Part 2 is what I ask and **who can answer it**, Part 3 is what I propose changing in someone else's code.

Deliberate exception to this repo's linking rule. The defect table in Part 1 is spelled out in full rather than linked, because this file gets read in a meeting by someone who will not follow links. It was cut from PUBLICATION_MATERIALS.md § Failure modes, which is the source of truth — if the two ever disagree, that file wins. Do not "fix" this into a link.

Replaces the old questions-for-malachi.md, meetings/, and the three Jul-24 question lists. When an answer arrives: the decision goes to decisions.md, the question is struck from here.

Part 1 — Where the work stands

The assignment, and what came back

Port the SciOptControlToolkit RL algorithms to PyTorch, TD3 first, validate on Gymnasium. Done, in a clone on branch develop-dw.

- **TorchTD3-v0 is written, registered and tested** — agents/torch_td3.py, models/torch_models.py, core/torch_model_core.py, cfgs/torch_td3.cfg. Commits ee4ced6, 61646af, 5c928ff.
- **82 tests**, all passing. The 47 on the critic and agent were then **mutation-tested**: three defects injected one at a time (minimum → maximum in the clipped double-Q, (1-dones) → dones, tau swapped in soft_update) and caught by 3, 5 and 1 tests respectively. "N tests pass" is weak evidence; "here is what the suite catches" is the claim.
- **Matched against Keras on four Gymnasium environments** — ahead on two (Pendulum-v1, Hopper-v5), behind on one (FetchReachDense-Flat-v0), tied on one (MountainCarContinuous-v0), where both agents reproduce TD3's canonical hard-exploration failure identically. That mixed pattern *is* the evidence for a faithful port; a clean sweep would more likely mean a changed hyperparameter. Numbers and logdirs in PUBLICATION_MATERIALS.md.
- **Additive only: 1,376 insertions, 0 deletions.** No existing toolkit file changed behaviour. That was a deliberate decision (decisions.md, 2026-08-17) and it is why Part 3 exists as *proposals* rather than as commits.
- **Driver and replay buffer deliberately untouched.** They do need work — but for **vectorization**, not for framework. Conflating "port to torch" with "support N parallel envs" is the mistake I avoided; scope in strategy.md.

Known weakness, stated before anyone asks: one seed per arm. The Keras arm cannot be re-seeded even in principle — it seeds from time.time_ns(). ≥3 seeds with reported variance is the top outstanding item.

The one structural finding that shaped the plan

Every research variant in the toolkit is a subclass of TD3 that overrides train_critic and changes almost nothing else:

<code>class KerasSINDyCriticTD3(KerasTD3)</code>	<code>keras_sindy_critic_td3.py:54</code>	
<code>class KerasUncertaintyTD3(KerasTD3)</code>	<code>keras_uncertainty_td3.py:46</code>	147 lines
<code>class KerasSINDyUncertaintyTD3(...)</code>	<code>keras_sindy_uncertainty_td3.py</code>	94 lines

So the SINDy-structured critic and the uncertainty-aware critic are **not separate algorithms — they are modifications of the Bellman-target computation inside TD3**. The most important property of any port is therefore a clean overridable `train_critic` seam, and "TD3 first" is a prerequisite rather than an ordering preference. This is also the cleanest possible framing for a Methods section: the baseline and every variant differ in exactly one function. **Please confirm I have read this right** — it is the assumption the whole design rests on.

Defects found while grounding the port

Each measured on a real run, not read off the page. Reported because the numbers this driver produces are the numbers a paper would cite.

Where	What	Consequence
<code>run_continuous.py:266</code>	<code>avg > best * (1 + threshold)</code> inverts sign for negative rewards. With <code>best = -845</code> the bar becomes <code>-887</code> , i.e. <i>below</i> best.	The "best model" checkpoint is not the best model. 10 of 10 eval points saved as improvements, one 98 points worse than an earlier one. <code>--load_model</code> cannot be trusted.
<code>run_continuous.py:264</code> vs <code>:281</code>	<code>nepisode_avg</code> counted in episodes in one place, eval points in the other. At the defaults the averaging window spans the whole run.	The reported average rises by dilution of the random-policy start, not because the policy improved: <code>-845 → -565</code> while the underlying evaluations bounced <code>-117 / -129 / -223 / -4 / -227</code> .
<code>keras_td3.py:177–202</code> , <code>run_continuous.py:64</code>	Three separate wall-clock seeds, and <code>env.action_space</code> never seeded at all.	No run is reproducible. Any single-run comparison is one unrepeatable sample.
<code>run_continuous.py:258</code>	The evaluation episode runs on the same env object as training.	Eval consumes and advances the training RNG stream, so the two are not statistically independent. Measured 170 points of evaluation spread across four same-seed runs of our own demo — and the evaluation number is the one that gets published.
<code>keras_td3.cfg</code>	<code>actor_update_freq / critic_update_freq</code> are read from config but absent from the file . Works only because the hardcoded fallback happens to be 2.	TD3's delayed policy update is not actually configurable, which breaks the toolkit's own no-hyperparameters-in-code rule.
<code>buffers/er.py:40–47</code>	1M rows preallocated regardless of fill, and <code>buffer.npy</code> rewritten every <code>ceil(nepisodes/10)</code> episodes.	80 MB written per snapshot always. A 15-episode smoke test wrote <code>≈720 MB</code> to persist 240 KB of experience.
<code>buffers/er.py:69</code>	<code>np.random.choice(..., replace=False)</code> materialises a full permutation: 82 <code>μs</code> at 5k rows, 16.2 ms at 1M .	Invisible on Pendulum, dominant once the buffer fills — at 1M rows index selection costs more than the gradient step it feeds. Any wall-clock comparison must state buffer occupancy.
<code>utests/</code>	<code>test_baselines.py</code> and <code>test_agents.py</code> train real agents into <code>results/</code> relative to the working directory.	11 run directories appeared in the shared results tree, indistinguishable from experiments. Anyone reporting from that tree can pick up a test artefact.
<code>TorchTD3.load()</code> (inherited)	Whole load wrapped in <code>try/except</code> that logs and continues with random weights .	An inference number can be a random policy's number with nothing in the output to say so. Now covered by two negative tests; the fallback itself is upstream and unchanged.

Part 2 — Questions to raise, grouped by who can answer

Ranked within each group by how much the answer changes what I build. Each is one line of ask, one line of what changes, and my recommendation where I have one.

A. Project scope — Malachi as PM. These block design, not just detail.

1. **What goes in Section 1.2, "Tasks"?** It is blank in the brief. → It is the single largest missing specification; success thresholds and the randomization envelope both hang off it.
2. **What is the success criterion — open to x degrees, what tolerance, and must the policy hold?** → Defines the termination condition and the success metric, which must stay separate from the training reward. *Existing code commits to a full sequence ending at 180°; I need that confirmed or replaced.*
3. **What is the randomization envelope** — approach angle, handle rotation, base pose, door friction/mass? → This is the generalization claim. Right now the task's `position_range` is `(0.0, 0.0)`, so nothing generalizes yet; see `learn/3-isaac-and-rl-games.md`. *My recommendation: fix an envelope now, however provisional, so evaluation can be built against it.*
4. **Are there concrete force/torque limits, and where do they come from?** → The brief requires "no brute-force pushing" and the existing task has **no force or torque anywhere** — not in observations, not in rewards, no contact sensor. A number turns that from a refinement into a requirement I can implement and test.
5. **Observation modality: privileged sim state, vision, or both?** → A major fork in effort. *Recommendation: privileged state for the baseline, vision as a later ablation — and say so in the paper rather than leaving it implicit.*
6. **Action space: joint targets, Cartesian/OSC, or torque — and is the mobile base in the RL loop?** → The existing code answers this *as built* (6 arm joint positions + binary gripper, base excluded) but not *as intended*. See the `base-dof0` reward anomaly in Part 2 § C.
7. **Is sim-to-real in scope at all?** → Determines how hard we push domain randomization, system ID and safety filters, and whether reset cost matters.
8. **Framework or task, and in what proportion for the paper?** → The stated goal is a modular test bed; a framework paper still needs a convincing task result. *Recommendation: build so the framework+task paper is guaranteed, the benchmark study is a section inside it, and the physics-informed result is upside claimed only if it lands —* `Learn/4-door-task.md` Part 3.
9. **Compute.** One RTX 5080 workstation caps `num_envs`. Is there a cluster, and is Azure realistic?
10. **Venue, deadline, co-authors.** → Sets how much goes into the framework versus the single task.

B. The RL toolkit — Malachi as its author.

1. **Which architecture for the Isaac Lab track?** `decisions.md` commits to `rl_games` PPO for the door task while we are adding TD3 to `jlab_opt_control`; those need reconciling. → *Recommendation (option B of three in `strategy.md`): `rl_games` stays the door-task baseline, the toolkit carries the method contribution, validated on `Gymnasium` first.*
2. **Is TD3 alone the deliverable, or do I port the SINDy and uncertainty variants next?** → Changes how much extensibility I build now. The `train_critic` seam is already in place for it.
3. **Is the SINDy / physics-informed direction a target contribution for the robotics paper, or specific to accelerators?** → It is the strongest novelty lever we have. If yes, what is the candidate library for door-opening — hinge angle, contact force, torque terms?
4. **A formulation discrepancy I want resolved.** Colen 2026's LC-TD3 surrogate predicts a physical *observable* (energy) with an explicit constraint penalty; the released `KerasSINDyCriticTD3` instead distills the *critic* into a sparse model. Which do you want carried forward, and does the observable-constraint variant live with the PACES code? → Decides what I actually implement.
5. **Is my understanding of SINDy right?** As I read it: the human supplies a *library* of candidate terms, sparse regression picks the few that matter, and the output is a short readable equation — which is the opposite of a PINN, where the equation is imposed by hand. Correct me if that is too simple.

6. **Can I get the paces package?** → It holds the CEBAF/LCLS surrogate envs; `run_continuous.py` imports it optionally and `ExaminePACESPolicy.ipynb` depends on it. Also: are there trained checkpoints I can load?
7. **Did prioritized replay meaningfully help on your problems, or is uniform ER the practical default?** → Decides whether PER is worth porting at all.
8. **Is the public repo the one to track, or is my snapshot fixed?** → Determines whether Part 3's proposals have anywhere to go.
9. **Differentiable-sim RL** — Rajput 2025 shows analytic-gradient methods winning at high dimension. In scope for the robot, or model-free first? *Recommendation: model-free first.*

C. The digital twin — prat615. Much of the old list is now answered by code.

`_reference/digital_twin_models-main/RL_CloudTesting/` answered, by being read: the door is a working articulated task, the gripper is driven through a single `finger_joint`, and there is a registered Isaac Lab env with a full PPO config. What the code did **not** answer:

1. **Which scene is canonical?** Our RL code uses `ENV_OT_UR_door/TrainingScene_flattened.usd`; the DT README points at `ENV_ESCBench/ESCBench_R0S_v6_140.usd`; the asset tree also ships `AIRoboticsLab.usd` and `experiment.usd`. → Deciding late means redoing scene paths.
2. **Gripper fork: 2F-85 or 2F-140?** RL code uses 2F-85; the DT README says current is 2F-140 with a v6 Ridgeback. → Redoing grasp tuning is the cost of getting this wrong.
3. **Is the mobile base drivable in physics?** `script_guided_base_dof0_profile_reward` (weight 0.4) shapes a base degree of freedom the action space does not contain. Either the base is actuated somewhere I have not found, or that term rewards something the policy cannot affect.
4. **Are mass, inertia and friction realistic or placeholder**, and which bodies are unreliable? → Bears directly on the anti-reward-hacking requirement.
5. **Is there a wrist F/T sensor modelled**, or do I read joint efforts / PhysX contact forces? → The force/torque requirement has to be implemented against something.
6. **Mimic or closed-loop gripper physics** for contact-rich grasping in 5.1, and are the `Defeatured_2F_85_*` collision meshes already wired into the recommended variant?
7. **Has this been run at `num_envs > 1`**, and is a stripped-down training scene (dropping the 59 MB room) available or wanted? → Directly caps throughput on one GPU.
8. **Is there a git repo to track** rather than a one-off zip, and are updates coming?
9. **What is the WidowX-AI teleop rig for** — demonstrations we could warm-start from, or a separate task?

D. Team-wide

1. **Who owns which layer?** Alvika leads RL, I support, prat615 owns the twin, the toolkit is Malachi's. Worth stating out loud so the env-definition boundary is not owned twice.
2. **Reproducibility as a team rule.** Fixed seeds, ≥ 3 seeds, reported variance, evaluation env seeded independently of training. → *Recommendation: agree this before results are generated, not after. The 170-point evaluation spread in Part 1 is what it costs to learn this later.*

Already answered — by our own work, not by a person

Kept so they stop being re-asked.

Question	Settled by
Shared or separate optimizers for the two critics?	Separate, one per critic — a deliberate deviation : Keras uses one Adam over both, which pools moment estimates across two networks TD3 needs to be independent estimators. <code>decisions.md</code> 2026-08-17. <i>Still worth your sign-off, since it is one of exactly three named differences between the arms.</i>
Is the wall-clock seeding intentional / must I copy it?	No. <code>seed</code> is a <code>cfg</code> key in <code>torch_td3.cfg</code> and the torch agent is reproducible. <code>decisions.md</code> 2026-08-17.
Does the dependency set really need Python 3.13, as <code>env.yaml</code> pins?	No — 3.11.9 resolves everything, TF 2.21.0 and torch 2.13.0+cpu in one env. <code>environment.md</code> . (<code>pysindy</code> is installed nowhere, which is why three notebooks do not run.)
Should <code>memory()</code> and <code>buffer</code> be in the agent ABC?	Moot for us: the real interface was derived from the driver and the tests, and is pinned by 82 of them. <code>core/agent_core.py</code> remains narrower than reality — worth mentioning, not worth blocking on.
Which env exercises genuine terminated?	Hopper-v5 of the four; and the door task does, via <code>sequence_complete</code> . Three of four Gymnasium envs never terminate, which is why mutation testing was necessary.
Is the twin asset-only?	No. <code>RL_CloudTesting/</code> is a working Isaac Lab door task with <code>rl_games</code> PPO, found 2026-08-03.

Part 3 — Upstream proposals

Three concrete patches. All are in shared driver or buffer code, so I am proposing rather than committing — the port was deliberately additive (1,376 insertions, 0 deletions). **Ask: may I submit these?**

1. Sign-safe checkpoint threshold — `drivers/run_continuous.py:266`, one line.

```
# current — for best < 0 this LOWERS the bar by `threshold`
if avg_inference_reward > best_avg_inference_reward * (1 + model_save_threshold):
# proposed
if avg_inference_reward > best_avg_inference_reward + model_save_threshold *
abs(best_avg_inference_reward):
```

Both Pendulum and accelerator tuning-error rewards are negative, so the current form admits results up to 5 % worse on exactly the tasks this toolkit is for. General rule worth adopting: relative thresholds must be written sign-safe.

2. `nepisode_avg` unit mismatch — :264 against :281. The averaging window is expressed in episodes but applied to a list appended once per `inference_interval`. Either divide by the interval or document the unit as eval points; as it stands the "recent average" spans the whole run and rises by dilution. Not a one-liner in intent even if it is in diff size — it changes what every past number meant, which is why it needs your call and not mine.

3. FlattenObservation for Dict observation spaces — :191, three lines. All 217 gymnasium-robotics ids (Fetch, Maze, Hand, Kitchen) publish a `Dict` space whose `.shape` is `None`, so `env.observation_space.shape[0]` raises `TypeError` before training starts. We currently work around it with an additive registry factory (`envs/robotics_flat.py`); the three-line wrapper in the driver would make every goal-conditioned env work for everyone. Caveat to state: flattening concatenates in **sorted key order**, and it is necessary but not sufficient — `FetchPush/Slide/PickAndPlace` still need `HER`.

Also worth raising, no patch offered: `buffers/er.py:69`'s `replace=False` (16.2 ms per sample at 1M rows; `replace=True` is flat at ~9 μs and is what every reference TD3 does), and that `pytest utests/` writes real training runs into `results/`.